

Registration App: User Guide

A beginner-friendly guide to what we built, the technology behind it, and how every piece works. It covers both the **theory** (why things are done this way) and the **technical** side (the actual code and commands).

1. What We Built

A small web application where a user can register their **name, place and phone number**, then view, edit and delete those registrations. Data is stored permanently in a MySQL database.

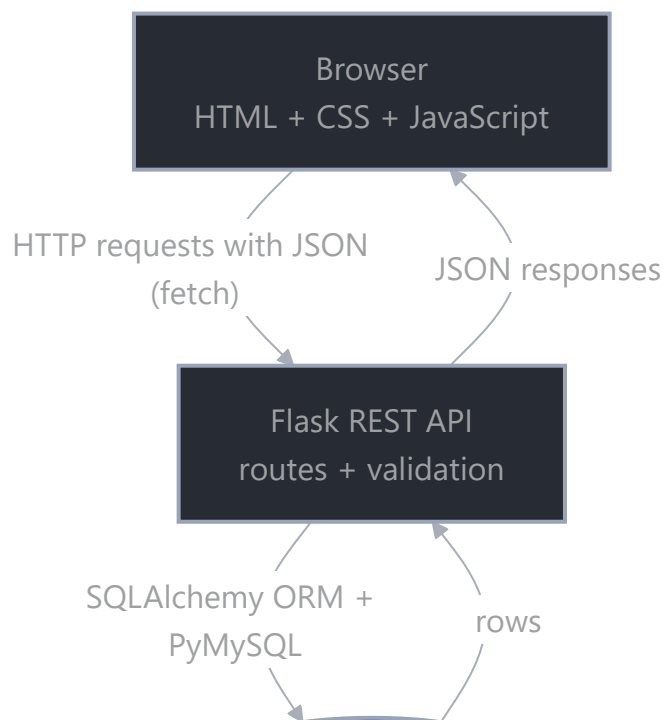
It supports the four CRUD operations:

Operation	Meaning	HTTP method	Example endpoint
Create	Add a new record	POST	/api/registrations
Read	Fetch one or all records	GET	/api/registrations , /api/registrations/1
Update	Change an existing record	PUT	/api/registrations/1
Delete	Remove a record	DELETE	/api/registrations/1

The project is deliberately simple: no testing, Docker or CI/CD yet. Those are planned as later stages, and the structure is kept clean so they can be added without rewriting anything.

2. Architecture

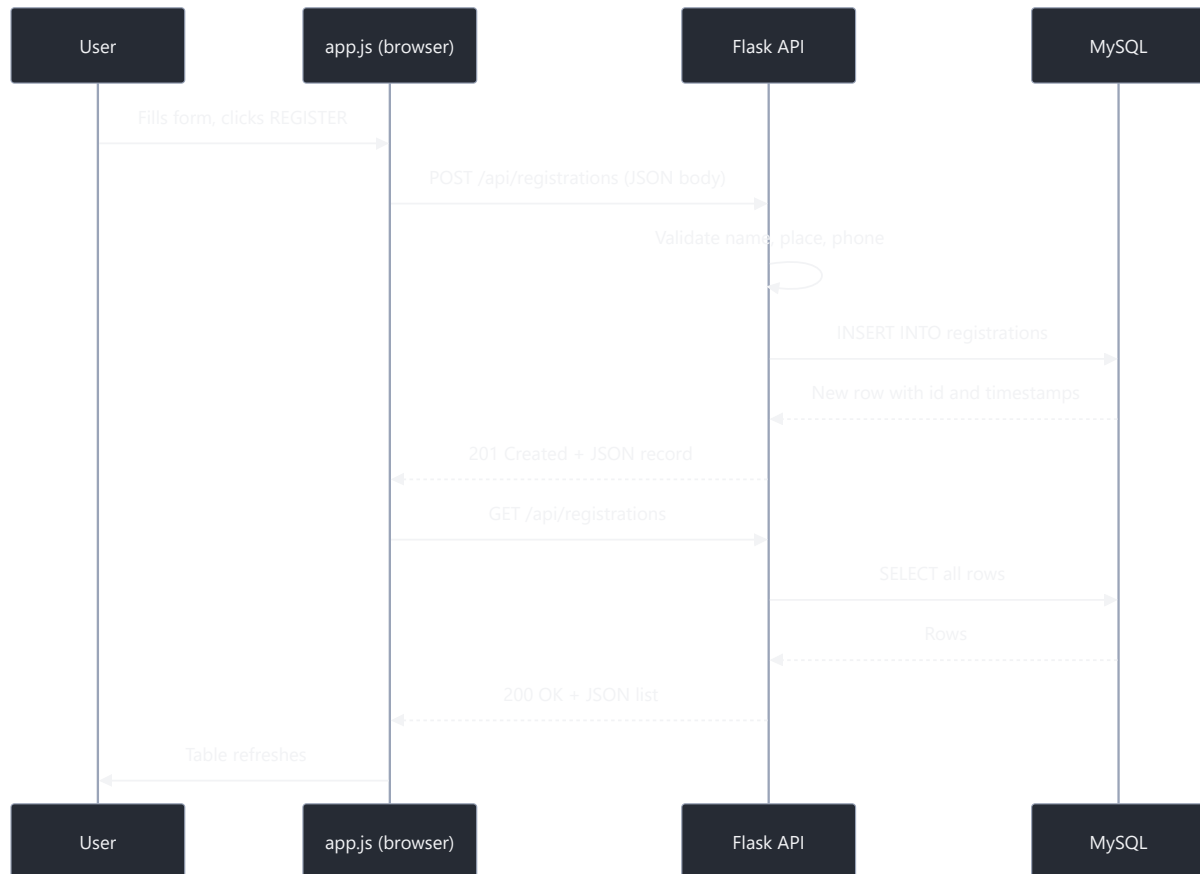
The app has three layers. Each layer only talks to its neighbour.





Theory: why layers? Separating the interface (browser), the logic (Flask) and the storage (MySQL) means you can change one without breaking the others. For example, you could replace the HTML page with a mobile app and the API and database would not need to change. This is called **separation of concerns**.

What happens when you click REGISTER



3. Technologies Used (and Why)

Technology	Role	Why we chose it
Python 3	Programming language for the backend	Readable, beginner-friendly, huge ecosystem
Flask	Web framework	Small and simple; gives you routing and JSON responses without extra weight
SQLAlchemy (via Flask-SQLAlchemy)	ORM (Object-Relational Mapper)	Lets you work with Python classes instead of writing raw SQL
PyMySQL	MySQL driver	The pure-Python library that actually talks to MySQL

cryptography	Support library for PyMySQL	MySQL 8 uses a secure login method (<code>caching_sha2_password</code>) that PyMySQL needs this for
python-dotenv	Loads the <code>.env</code> file	Keeps passwords out of the source code
MySQL 8	Relational database	Reliable, widely used, stores data permanently
HTML / CSS / JavaScript	Frontend	No framework needed: HTML is structure, CSS is looks, JS is behaviour
fetch()	Browser API for HTTP calls	Lets the page talk to the Flask API without reloading

4. Core Concepts (Theory)

4.1 Client-server model

The **client** (your browser) asks for things; the **server** (Flask) answers. The browser never touches the database directly. It always goes through the server, which decides what is allowed.

4.2 REST API

REST is a convention for designing APIs around **resources** (here: `registrations`) and standard HTTP methods. The URL names the thing, and the method says what to do with it.

- `GET /api/registrations` means "give me all registrations".
- `DELETE /api/registrations/5` means "delete registration 5".

4.3 HTTP status codes

Every response carries a number that tells the client what happened.

Code	Meaning	Where we use it
200	OK	Successful read, update or delete
201	Created	Successful create
400	Bad Request	Missing field, too-long value, invalid ID
404	Not Found	ID is valid but no such record
500	Server Error	Database or unexpected failure

4.4 JSON

The format used to send data between browser and server. Example:

```
{ "name": "Harish", "place": "Hyderabad", "phone": "9876543210" }
```

4.5 ORM (Object-Relational Mapping)

Instead of writing `INSERT INTO registrations ...` by hand, we define a Python class `Registration`.

Each class attribute maps to a table column, and each object maps to a row. SQLAlchemy translates between them. It also protects against **SQL injection**, because values are passed as parameters rather than glued into SQL text.

4.6 Blueprints

A Flask Blueprint groups related routes in their own file. All `/api/registrations` routes live in `routes/registration_routes.py` and are plugged into the main app, which keeps `app.py` small.

4.7 Environment variables and secrets

Passwords must never be written into code (they would end up in Git). We store them in a `.env` file, which `.gitignore` excludes, and read them at runtime. `.env.example` shows which variables are needed without containing real secrets. This is a standard practice from the "twelve-factor app" approach.

4.8 Principle of least privilege

The app connects as `registration_user`, not `root`. That user can only `SELECT`, `INSERT`, `UPDATE` and `DELETE` on one table. If the app were ever compromised, the attacker could not drop databases or read other data.

4.9 Why phone is VARCHAR, not INT

Phone numbers are identifiers, not quantities. You never do arithmetic on them. As integers they would lose leading zeros and could not hold `+91` prefixes.

4.10 Validation

Never trust input from the browser. The server checks every request, even though the form also has basic checks, because anyone can call the API directly (for example with `curl`).

4.11 XSS protection

If a user registered with the name `<script>...</script>` and we inserted it into the page as HTML, it could run code in other visitors' browsers. Our JavaScript uses `textContent` instead of `innerHTML` when filling the table, so such input is displayed as plain text.

5. Project Structure

```

registration-app/
├── app.py                # Creates the Flask app, /health, error handlers
├── requirements.txt      # Python dependencies (pinned versions)
├── README.md            # Short setup reference
├── .env.example         # Template of required environment variables
├── .gitignore           # Keeps .env, venv, caches out of Git
├── config/
│   └── database.py      # Builds the DB connection string; creates `db`
├── models/
│   └── registration.py  # The Registration table as a Python class
├── routes/
│   └── registration_routes.py # The CRUD API endpoints and validation
├── templates/
│   └── index.html       # The page (form + table)
├── static/
│   ├── css/style.css    # Styling
│   └── js/app.js        # Frontend logic using fetch()

```

6. Implementation Walkthrough

6.1 Database

```
CREATE TABLE registrations (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  place       VARCHAR(100) NOT NULL,
  phone       VARCHAR(20)  NOT NULL,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

- `AUTO_INCREMENT` makes MySQL assign each new `id` automatically.
- `NOT NULL` makes the database itself reject empty values.
- `created_at` is filled on insert; `updated_at` refreshes automatically on every update. MySQL does both, so our Python code never sets timestamps.

The app does **not** create tables itself. We run the SQL once by hand, which keeps the app's database user limited to data operations only.

6.2 Configuration (`config/database.py`)

Reads `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD` and builds a connection string:

```
mysql+pymysql://user:password@host:port/database?charset=utf8mb4
```

- `mysql+pymysql` tells SQLAlchemy to use the MySQL dialect with the PyMySQL driver.
- `quote_plus` makes sure special characters in a password (like `@` or `#`) do not break the URL.
- It also creates the shared `db = SQLAlchemy()` object that the model and app both import.

6.3 Model (`models/registration.py`)

```
class Registration(db.Model):
    __tablename__ = "registrations"
    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    name = db.Column(db.String(100), nullable=False)
    ...
    def to_dict(self):
        return {"id": self.id, "name": self.name, ...}
```

The class mirrors the table. `to_dict()` converts a row into a plain dictionary so Flask can send it as JSON (timestamps are converted to ISO text).

6.4 Routes (`routes/registration_routes.py`)

Function	Route	What it does
<code>create_registration</code>	<code>POST /api/registrations</code>	Validates the JSON body, inserts a row, returns 201

<code>get_registrations</code>	<code>GET /api/registrations</code>	Returns every row, ordered by id
<code>get_registration</code>	<code>GET /api/registrations/<id></code>	Returns one row or 404
<code>update_registration</code>	<code>PUT /api/registrations/<id></code>	Validates, updates the row, returns it
<code>delete_registration</code>	<code>DELETE /api/registrations/<id></code>	Deletes the row, returns a message

Two helper functions do the shared work:

- `parse_id` accepts only positive whole numbers. Anything else (like `abc` or `-5`) returns 400 "Invalid ID". We take the ID as a string on purpose. If we had used Flask's `<int:id>`, a bad ID would produce Flask's own HTML 404 page instead of our JSON error.
- `validate_payload` checks that the body is JSON, that `name`, `place` and `phone` are non-empty text, trims whitespace, and enforces the maximum lengths (100, 100, 20). It returns either cleaned data or an error message.

6.5 Application setup (`app.py`)

- `create_app()` is the "application factory" pattern: it builds and configures the app, connects the database, and registers the blueprint. This pattern makes later testing and Docker use easier.
- `pool_pre_ping` makes SQLAlchemy test a database connection before using it, so stale connections (for example after MySQL restarts) are replaced instead of causing errors.
- `GET /` serves the HTML page. `GET /health` returns `{"status": "UP"}`, which is the kind of endpoint load balancers and orchestrators call later.
- **Error handlers** turn failures into simple JSON:

Situation	Response
Database unreachable (<code>OperationalError</code>)	500 Database connection failed
Any other database error	500 A database error occurred
Normal HTTP errors (404, 405...)	JSON with the error name
Anything unexpected	500 Internal server error

Details are written to the server log only, so passwords and internals are never exposed to the browser. On a database error the session is rolled back so the next request starts clean.

6.6 Frontend

`index.html` contains the form (Name, Place, Phone, REGISTER and a hidden CANCEL button) and an empty table body that JavaScript fills.

`app.js` works like this:

- `request(url, options)` wraps `fetch()`. It sends JSON, parses the reply, and throws an error carrying the API's message if the status is not OK.
- `loadRegistrations()` calls `GET /api/registrations` and passes the result to `renderTable()`, which builds the table rows with DOM methods.
- **Register / Update:** the form's submit handler checks the variable `editingId`. If it is `null` it sends a

POST ; if it holds a number it sends a PUT . Afterwards it resets the form and reloads the table.

- **Edit:** startEdit(id) fetches that record, fills the form, sets editingId , and changes the button to UPDATE. CANCEL resets everything.
- **Delete:** shows a confirm() dialog, sends DELETE , then reloads the table.
- Errors from the API appear in a message box above the table.

7. API Reference

Base URL: http://127.0.0.1:5000

Method	Endpoint	Body	Success
GET	/health	none	200 {"status": "UP"}
POST	/api/registrations	name, place, phone	201 + record
GET	/api/registrations	none	200 + list
GET	/api/registrations/<id>	none	200 + record
PUT	/api/registrations/<id>	name, place, phone	200 + updated record
DELETE	/api/registrations/<id>	none	200 {"message": "Registration deleted"}

Example record returned by the API:

```
{
  "id": 1,
  "name": "Harish",
  "place": "Hyderabad",
  "phone": "9876543210",
  "created_at": "2026-09-30T17:45:12",
  "updated_at": "2026-09-30T17:45:12"
}
```

Example error:

```
{ "error": "name is required" }
```

Testing with curl:

```
# Create
curl -X POST http://127.0.0.1:5000/api/registrations \
  -H "Content-Type: application/json" \
  -d '{"name": "Harish", "place": "Hyderabad", "phone": "9876543210"}'

# Read all / one
curl http://127.0.0.1:5000/api/registrations
curl http://127.0.0.1:5000/api/registrations/1
```

```
curl http://127.0.0.1:5000/api/registrations/1
```

```
# Update
```

```
curl -X PUT http://127.0.0.1:5000/api/registrations/1 \
  -H "Content-Type: application/json" \
  -d '{"name": "Harish Gundimeda", "place": "Hyderabad", "phone": "9876543210"}'
```

```
# Delete
```

```
curl -X DELETE http://127.0.0.1:5000/api/registrations/1
```

Try bad input too (an empty name, or ID abc , or ID 9999) and confirm you get 400, 400 and 404.

8. Setup and Running (Windows, Git Bash)

1. **Install** Python 3 and MySQL 8, and make sure the MySQL service is running.
2. **Create the database, table and user.** Open the MySQL Command Line Client, log in as root, and run:

```
CREATE DATABASE IF NOT EXISTS registration_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE registration_db;

CREATE TABLE IF NOT EXISTS registrations (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  place       VARCHAR(100) NOT NULL,
  phone       VARCHAR(20)  NOT NULL,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE USER IF NOT EXISTS 'registration_user'@'localhost' IDENTIFIED BY 'your_password';
GRANT SELECT, INSERT, UPDATE, DELETE ON registration_db.registrations TO 'registration_user'@'
FLUSH PRIVILEGES;
```

3. **Create a virtual environment** in the project folder and activate it:

```
python -m venv venv
source venv/Scripts/activate
```

A virtual environment is a private folder of Python packages for this project only, so versions do not clash with other projects. On Linux/macOS the activate path is `venv/bin/activate`.

4. **Install dependencies:** `pip install -r requirements.txt`
5. **Configure:** `cp .env.example .env`, then edit `.env` so `DB_PASSWORD` matches the MySQL user's password.
6. **Run:** `python app.py`, then open `http://127.0.0.1:5000`

9. Troubleshooting

Symptom	Cause	Fix
<code>venv/bin/activate: No</code>	<code>venv</code> not created, or on	Run <code>python -m venv venv</code> ; on

such file	Windows	Windows use <code>source venv/Scripts/activate</code>
<code>python3</code> not found (Windows)	Windows uses <code>python</code> or <code>py</code>	Use <code>python</code> or <code>py</code>
Database connection failed in the page	Wrong password in <code>.env</code> , MySQL not running, or user/DB not created	Check <code>.env</code> , start the MySQL service, re-run the setup SQL
Access denied for user in the terminal log	Password mismatch	Make <code>.env</code> match the <code>CREATE USER</code> password (or <code>ALTER USER</code> to change it)
Table 'registrations' doesn't exist	Table was never created	Run the <code>CREATE TABLE</code> statement inside <code>registration_db</code>
<code>cryptography</code> package is required	Dependency missing	<code>pip install -r requirements.txt</code> inside the activated <code>venv</code>
Port 5000 already in use	Another program uses it	Change the port in the last line of <code>app.py</code>
Changes to <code>.env</code> ignored	App reads it only at start	Stop and restart <code>python app.py</code>

10. Security Notes

- Credentials come from environment variables; `.env` is never committed.
- The app uses a limited MySQL user, not root.
- SQLAlchemy uses parameterized queries, which protects against SQL injection.
- Input is validated on the server; the frontend escapes output with `textContent`.
- Error messages never include passwords or stack traces.
- `debug=True` in `app.py` is for local development only. It must be turned off before any real deployment, because debug mode can expose an interactive console.
- Flask's built-in server is for development. Production deployments use a server like Gunicorn.

11. Limitations and Next Stages

Intentionally left out for now: automated tests, duplicate-phone checks, phone-format validation, authentication, and pagination.

The structure is ready for the planned stages:

Stage	What it adds	Why the current structure helps
Testing (pytest)	Automated tests of the API	<code>create_app()</code> factory and a separate config make it easy to test
Docker / Compose	Package app and MySQL as containers	Settings already come from environment variables

CI/CD (Jenkins, GitHub Actions)	Automatic build, test, deploy	requirements.txt is pinned and .gitignore is clean
Cloud, Kubernetes, Terraform	Deployment and infrastructure as code	/health endpoint is ready for health probes
Monitoring	Metrics and alerts	Server logging is already in place

12. Glossary

Term	Meaning
API	A set of URLs a program can call to use another program's features
CRUD	Create, Read, Update, Delete
Endpoint	One URL + HTTP method combination, such as GET /health
ORM	Tool that maps database tables to Python classes
Virtual environment	Isolated Python packages for one project
Environment variable	A setting stored outside the code, such as a password
Blueprint	A Flask way to group related routes
Primary key	The column that uniquely identifies each row (id)
Idempotent	Repeating the request gives the same result (PUT and DELETE are meant to be)

